EMBEDDED PREP · SUBJECT 1

Microcontrollers

ARM Cortex-M — Complete Notes

Question-first notes for interview recall & bench reference
Concrete examples target the STM32F746 (Cortex-M7)

Core · GPIO/EXTI · Timers · ADC/DAC · Memory/DMA/Flash · Clock & Power · Comm Peripherals · Dev & Tooling

Master Index

Microcontrollers (ARM Cortex-M) — Master Index

> Subject 1 of the embedded-prep set. Notes are **question-first**: question → tight answer → **gotcha/failure mode** → snippet/self-test. Optimized for **both** interview recall and bench reference. Concrete examples target the **STM32F746 (Cortex-M7)**; the core concepts transfer to any Cortex-M (and largely to RISC-V).

The files

#	File	Covers	Heaviest interview hits
01	`01_microcontrollers_arm_cortex.md`	Core: families, register set, modes, memory map, boot/vector table, NVIC/exceptions, faults, M7 caches/TCM, MPU, FPU, CMSIS, debug intro, RISC-V contrast, C gotchas	Vector table first word, NVIC priority shift, `volatile`, DMA+cache
02	`02_gpio_and_exti.md`	GPIO modes/AF, push-pull vs open-drain, BSRR atomic set/reset, EXTI, debouncing	BSRR vs ODR race, I2C open-drain, EXTI pending-flag storm
03	`03_timers_pwm_watchdog_rtc.md`	Timer families, PSC/ARR, PWM, input capture, encoder, advanced-timer dead-time, IWDG/WWDG, RTC	N−1 off-by-one, 2× timer clock, advanced-timer MOE
04	`04_adc_dac.md`	SAR fundamentals, **SAR vs sigma-delta**, sampling time, conversion modes, ADC+DMA, Vref/calibration, aliasing/oversampling, DAC	Sampling time vs source impedance, timer-triggered+DMA, F7 cache/DTCM
05	`05_memory_dma_flash.md`	Memory map/types, Flash program/erase/wait-states, linker sections, **DMA** modes, **F7 DTCM-not-DMA-accessible**, cache coherency	Erase granularity, WS ordering, DTCM/DMA trap, clean/invalidate
06	`06_clock_and_power.md`	Clock sources, PLL, clock tree, **CSS**, voltage scaling/over-drive, **BOR vs PVD**, reset flags, low-power modes	Clock-up ordering, CSS NMI, BOR vs PVD, SLEEPDEEP
07	`07_comm_peripherals_config.md`	**Config side only**: USART/SPI/I2C/CAN registers, clocking, DMA/IRQ pairing, USB/Eth/SDMMC/I2S highlights	UART baud error, F7 SPI FRXTH, F7 I2C TIMINGR, CAN bit-timing/sample point
08	`08_dev_and_tooling.md`	HAL/LL/CMSIS/register decision table, GCC toolchain/linker/make, SWD debug, breakpoints/watchpoints, CRC peripheral, bootloaders/IAP	Float ABI match, BP vs WP, CRC reversal, VTOR/MSP jump

Scope decisions (so nothing is duplicated)

- **Peripheral *config*** (registers, clocking, DMA pairing, bring-up) lives **here** (file 07 + each peripheral file).
- **Protocol *theory*** (frame formats, addressing, arbitration, signal levels, your ADS131M08 frame structure, CAN payloads) is deferred to the **Communication Protocols** subject.

- **RTOS** specifics (PendSV context switch, FreeRTOS priority config) get a light touch here and the full treatment in the **RTOS** subject.

How to revise

1. **Passive pass (anytime):** open one file, cover the answers, recall each out loud, peek to check. One file per sitting keeps topic boundaries clean for spaced repetition.
2. **Rapid-fire pass:** each file ends with a numbered Q&A — use these as flashcards before an interview.
3. **Active/coding pass (daily):** each file ends with "daily-snippet candidates." Retype one from memory, compile/flash (or run under QEMU/Renode when away from the board), don't copy-paste.

Strongest personal-leverage notes

These tie to work you've already done — write/expand them from your own experience, they'll stick hardest: the **DMA+cache / DTCM trap** (04, 05) and **SPI config** (07) against your ADS131M08 driver; **CAN bit-timing** (07) against your DRDO CAN work; **register-level vs HAL** (08) as your closed Feb gap.

Deepening with a book (next options)

Per-file footers list sources. Best targets: **Yiu, *Definitive Guide to Cortex-M7*** for 01/08; **STM32F7 Reference Manual (RM0385/RM0410)** for register-level depth in 02–07; the **ADS131M08 datasheet** for the sigma-delta counterpart to 04. Upload one and say which file to deepen — I'll add exact bitfields and worked examples rather than starting fresh.

Part 1 — Core / ARM Cortex Architecture

Microcontrollers — ARM Cortex-M

> **How to use this file.** Each topic is written question-first: a question you'd get asked (interview) or hit in practice, a tight answer in plain language, the **gotcha / failure mode** (the part that actually bites you), and where useful a minimal snippet or register note. Cover the answer, recall it out loud, then check. Hardware references target the **STM32F746 (Cortex-M7)** where concrete examples help.

0. Map of the territory

The Cortex-M architecture is the part of "microcontrollers" that is genuinely *standardized* — ARM defines the core, the exception model, the memory map, and the debug interface, and every vendor (ST, NXP, TI, Nordic…) wraps their own peripherals around it. So learning Cortex-M deeply transfers across every STM32, every nRF, every Kinetis. The vendor-specific part is the peripherals + clock tree + memory sizes; the core part below is the same everywhere.

1. The Cortex-M family — who's who

Q: What are the main Cortex-M cores and how do they differ?

- **M0 / M0+** — smallest, ARMv6-M, subset of Thumb-2, no/limited NVIC priorities, M0+ adds a faster 2-stage pipeline and optional MPU. Think ultra-low-power, cost-sensitive.
- **M3** — ARMv7-M, full Thumb-2, full NVIC, bit-banding, divide instruction, MPU optional. The classic workhorse.
- **M4** — M3 + DSP instructions (SIMD, MAC) + optional single-precision FPU. Used for motor control, audio, light signal processing.
- **M7** — ARMv7-M, superscalar (dual-issue) 6-stage pipeline, caches (I/D), Tightly-Coupled Memory (TCM), optional double-precision FPU. High-performance. **Your STM32F746 is an M7.**
- **M23 / M33** — ARMv8-M, add **TrustZone** (secure/non-secure worlds). M33 is the security- oriented modern mainstream.

Gotcha: "Cortex-M4 has an FPU" is only sometimes true — the FPU is *optional* and single-precision only. A part labelled Cortex-M4 vs Cortex-M4F tells you. Doing `double` math on an M4F still hits software emulation because the hardware FPU is single-precision. M7 can have a double-precision FPU but, again, optional.

2. Programmer's model — registers & modes

Q: Describe the Cortex-M register set.

- **R0–R12** general purpose (R0–R7 "low", R8–R12 "high").
- **R13 = SP** — stack pointer, *banked*: **MSP** (Main) and **PSP** (Process).
- **R14 = LR** — link register, holds return address; in exceptions holds the special **EXC_RETURN** value.
- **R15 = PC** — program counter.
- **xPSR** — program status: APSR (flags N,Z,C,V), IPSR (current exception number), EPSR (Thumb bit, IT state).
- Special registers: **PRIMASK / FAULTMASK / BASEPRI** (interrupt masking), **CONTROL** (privilege level + which SP is active in Thread mode + FPU active).

Q: Thread mode vs Handler mode? MSP vs PSP?

- **Handler mode** — runs exception/interrupt handlers, always privileged, always uses **MSP**.
- **Thread mode** — runs your normal application code; can be privileged or unprivileged, can use MSP *or* PSP (selected by CONTROL bit).
- Bare-metal apps usually run everything on MSP. **An RTOS uses PSP for task stacks and MSP for the kernel/ISRs** — this is why each task gets its own stack and the kernel doesn't corrupt them.

Gotcha (ties to RTOS): when FreeRTOS does a context switch in PendSV, it's saving/restoring the *PSP* stack frame. If you ever see stack corruption only under an RTOS, suspect task stack size or the PSP frame, not MSP.

---

3. The fixed memory map

Q: Sketch the Cortex-M memory map.

It's architecturally fixed (vendors place real memory inside these windows):

Region	Range (typical)	Use
Code	`0x0000_0000` – `0x1FFF_FFFF`	Flash, vector table at boot
SRAM	`0x2000_0000` – `0x3FFF_FFFF`	RAM (bit-band region in low 1MB)
Peripheral	`0x4000_0000` – `0x5FFF_FFFF`	Peripheral registers (bit-band)
External RAM	`0x6000_0000` – `0x9FFF_FFFF`	FMC / external memory
External dev	`0xA000_0000` – `0xDFFF_FFFF`	External peripherals
Private Periph (PPB)	`0xE000_0000` – `0xE00F_FFFF`	NVIC, SCB, SysTick, debug

Q: What is bit-banding?

A region of SRAM/peripheral space is aliased so each *bit* maps to its own *word* address. Writing 1 to the alias word sets just that bit atomically — no read-modify-write, so no interrupt-race on a shared register. M3/M4 have it; **M7 does not** (it relies on other atomicity mechanisms). Don't assume bit-banding exists on your F746.

---

4. Boot & reset sequence

Q: What exactly happens from power-on to `main()`?

1. Core comes out of reset, reads **two words from the start of the vector table**: - word 0 → **initial Main Stack Pointer (MSP)** - word 1 → **reset vector** (address of `Reset_Handler`), with bit0=1 (Thumb).
2. `Reset_Handler` (startup code) runs: copies `.data` from Flash to RAM, zeroes `.bss`, optionally sets up clocks (`SystemInit`), then calls `__libc_init_array` (C++ ctors) and `main`.
3. The CPU starts in **Thread mode, privileged, using MSP**.

Gotcha #1 (classic interview): the *first* word of the vector table is **not code, it's the stack pointer value**. People draw it as "reset vector first" — wrong, SP is first.

Gotcha #2 (real bug): forgetting to init `.data`/`.bss` (custom startup without the copy loop) → globals have garbage values, `static int x = 5;` reads as junk. If you've ever written startup from scratch, this is the bug.

Q: What is VTOR and why relocate the vector table?

`SCB->VTOR` points the core at the vector table. Bootloaders use this: the bootloader lives at `0x0800_0000`, then jumps to the app at e.g. `0x0800_8000` and sets `VTOR` so the app's own interrupt vectors are used. Forgetting to set VTOR after a bootloader jump → interrupts vector into the *bootloader's* handlers. Common firmware-update bug.

5. Exceptions & the NVIC — the heart of it

Q: Walk through the Cortex-M interrupt mechanism.

- **NVIC** (Nested Vectored Interrupt Controller) handles all external interrupts + some system exceptions; it's *part of the core*, not a vendor peripheral.
- On an interrupt the hardware **automatically stacks** 8 registers (R0–R3, R12, LR, PC, xPSR) — this is why ISRs can be plain C functions with no special prologue.
- LR is loaded with **EXC_RETURN** (a magic value like `0xFFFFFFF9`) encoding which mode/stack to return to. Returning by loading EXC_RETURN into PC triggers automatic unstacking.

Q: Priority, preemption, sub-priority?

- Lower number = higher priority. Priority is split by `PRIGROUP` into **preemption** bits and **sub-priority** bits.
- Preemption: a higher-preemption interrupt *interrupts* a running lower one (nesting).
- Sub-priority: only decides ordering between two *pending* interrupts of equal preemption — it does **not** cause preemption.

Gotcha (huge, RTOS-related): STM32 implements only the top **4 bits** of the 8-bit priority field. So priorities are 0, 16, 32… not 0,1,2. And FreeRTOS's `configMAX_SYSCALL_INTERRUPT_PRIORITY` must be set in these *shifted* terms — any ISR that calls a `…FromISR()` API must have a numerically *higher* priority value (lower urgency) than that threshold, or you get silent corruption. This is one of the most common FreeRTOS-on-STM32 bugs.

Q: Tail-chaining, late-arrival?

Performance optimizations the core does automatically: - **Tail-chaining** — if a second interrupt is pending when one finishes, the core skips the pop+push and jumps straight to the next handler (saves ~12 cycles). - **Late arrival** — if a higher-priority interrupt arrives *during* the stacking of a lower one, the core services the higher one first.

Q: Masking registers?

- **PRIMASK** — disable all maskable interrupts (`__disable_irq()`).
- **FAULTMASK** — also masks HardFault.
- **BASEPRI** — disable all interrupts *at or below* a given priority. This is what RTOS critical sections use, so high-priority ISRs still run.

6. Fault handling & debugging crashes

Q: What faults exist and how do you debug a HardFault?

- **HardFault** — catch-all; escalated-to fault.
- **MemManage** — MPU violation.
- **BusFault** — bad memory access (unmapped address, etc.).
- **UsageFault** — illegal instruction, divide-by-zero (if enabled), **unaligned access** (if trap enabled).

Debug recipe: 1. Read the fault status registers in the SCB: **CFSR** (`SCB->CFSR`), **HFSR**, and the fault address registers **MMFAR / BFAR**. 2. Recover the stacked PC/LR from the exception stack frame to find *where* it died. 3. A minimal HardFault handler that captures the stacked frame:

```
void HardFault_Handler(void) {
    __asm volatile (
        "tst lr, #4        \n"   // which stack? bit2 of EXC_RETURN
        "ite eq            \n"
        "mrseq r0, msp     \n"
        "mrsne r0, psp     \n"
        "b hardfault_report\n"   // r0 = faulting stack frame
    );
}
void hardfault_report(uint32_t *sf) {
    uint32_t stacked_pc = sf[6];  // R0,R1,R2,R3,R12,LR,PC,xPSR
    (void)stacked_pc;             // breakpoint here, inspect
}
```

Gotcha: the commonest cause of an immediate HardFault on boot is an **unaligned access** or a **null/garbage function pointer** (often an uninitialized callback). Second commonest: calling an RTOS or HAL function before the clocks/RTOS are initialized.

---

7. Clock tree (vendor side, but always asked)

Q: Trace the clock from oscillator to peripheral.

- Source: **HSI** (internal RC, fast to start, imprecise) or **HSE** (external crystal, precise).
- Into the **PLL** to multiply up to SYSCLK (e.g. 216 MHz on F746).
- SYSCLK → AHB prescaler → **HCLK** (core, memory, DMA).
- HCLK → APB1/APB2 prescalers → peripheral clocks (and timer clocks, which often run at 2× the APB clock when the APB prescaler ≠ 1).

Gotcha #1: a peripheral does nothing until its clock is enabled in `RCC->AHBxENR/APBxENR`. "My GPIO/UART is dead" → 90% of the time the RCC clock-enable bit is missing. Also there can be a required dummy read after enabling before the peripheral registers are writable.

Gotcha #2 (timer math): APBx timer clock is often `2×PCLKx`. Getting baud rates, PWM frequencies, or your DAQ sample timing wrong usually traces back to this 2× factor.

---

8. SysTick

Q: What is SysTick and who owns it?

A simple 24-bit down-counter inside the core (not a vendor timer), designed as the RTOS tick. Reload value = `(clock / tick_rate) − 1`. It generates the **SysTick exception** at the tick rate.

Gotcha: if you use an RTOS, **the RTOS owns SysTick** — don't also use it for HAL's `HAL_Delay`/`HAL_GetTick` driving its own SysTick callback, or pick a different timebase for HAL (STM32 lets you select a TIM as the HAL timebase). Double-owning SysTick = drifting ticks.

---

9. Cortex-M7 specifics (directly relevant to your F746 + DAQ work)

Q: What do caches and TCM change about how you write drivers?

The M7 adds I-cache and D-cache plus optional TCM (ITCM/DTCM — single-cycle, never cached). This buys big speed but introduces **cache coherency** problems with DMA.

Gotcha — the DMA + D-cache trap (this is *your* world at 8 kSPS):

- **MCU → peripheral (TX):** you fill a buffer, but the bytes may still sit in D-cache, not in the RAM the DMA reads. You must `SCB_CleanDCache_by_Addr()` before starting the DMA so RAM is up to date.
- **Peripheral → MCU (RX, e.g. your ADS131M08 samples landing via DMA):** DMA writes RAM directly, but the CPU may read a **stale cached copy**. You must `SCB_InvalidateDCache_by_Addr()` after the transfer completes, before reading the buffer.

- Addresses/sizes for these must be **32-byte aligned** (cache line size) or you corrupt adjacent data. Use `__attribute__((aligned(32)))` and round the length up.
- Pragmatic alternatives: place DMA buffers in a **non-cacheable MPU region**, or in **DTCM** (not cached, and conveniently fast) — many people just put DAQ ring buffers in DTCM to dodge the whole problem. Worth a line in your DAQ notes.

Q: Why is the FPU lazy-stacked?

The M7/M4 FPU has 16–32 extra registers (S0–S31). Stacking them on every interrupt is expensive, so the core uses **lazy stacking**: it reserves space but only actually saves FPU regs if the handler uses the FPU. Relevant to RTOS context-switch sizing.

---

10. Low-power modes

Q: Sleep vs Stop vs Standby?

- **Sleep** — core clock gated, peripherals + RAM alive, wakes on any interrupt. Fast.
- **Stop** — most clocks off, RAM/registers retained, wakes from EXTI/RTC. Big savings, slower wake.
- **Standby** — almost everything off, RAM lost (except backup domain), wakes ≈ reset. Lowest power.

Entered via `WFI` / `WFE` instructions plus power-control register bits. **Gotcha:** debuggers often can't keep a connection through Stop/Standby — "my debugger drops when I sleep" is expected, not a bug.

---

11. MPU (Memory Protection Unit)

Q: What does the MPU do (and not do)?

Defines a handful of regions with attributes (read/write/execute, privileged-only, cacheable/bufferable). It enforces access rules and triggers a MemManage fault on violation. It is **not** an MMU — no virtual memory, no address translation, fixed small number of regions.

Uses: catch stack overflows / null-pointer derefs, mark a region non-cacheable for DMA (see §9), sandbox unprivileged RTOS tasks.

---

12. CMSIS

Q: What is CMSIS?

ARM's **Cortex Microcontroller Software Interface Standard** — a vendor-neutral hardware abstraction for the *core* (not vendor peripherals): the `core_cm7.h` register structs (`SCB->`, `NVIC->`), intrinsics (`__disable_irq`, `__DSB`, `__WFI`), the `SystemInit` / `SystemCoreClock` convention, plus CMSIS-DSP and CMSIS-RTOS. Even register-level (no-HAL) code normally still uses CMSIS for the core definitions — that's the sweet spot of "bare-metal but not insane."

---

13. Debug & trace

Q: SWD vs JTAG? What's ITM/SWO?

- **JTAG** — older, 4–5 pins, daisy-chains multiple devices.
- **SWD** — ARM's 2-pin (SWDIO/SWCLK) serial debug, same capability for a single core. Standard now.
- **ITM + SWO** — single-wire trace output; `printf`-style debug over SWO without a UART.
- **Semihosting** — `printf` routed through the debugger to the host console; convenient but **halts the core on each call** — never leave it in production/timing-sensitive code.

---

14. RISC-V contrast (you listed it — know the talking points)

Q: How does RISC-V differ from ARM Cortex-M at a high level?

- **Open ISA** (no license fee) vs ARM's licensed cores.
- **Base + extensions** model: RV32I base, plus M (mul/div), A (atomic), F/D (float), C (compressed). You name a core by its extensions, e.g. RV32IMC.
- No fixed NVIC-style exception model standardized the way Cortex-M's is — interrupt controllers (CLINT/PLIC) vary more by implementation, so portability across RISC-V MCUs is currently looser than across Cortex-M.
- Concepts transfer directly: pipeline, traps/exceptions, memory-mapped peripherals, privilege levels (machine/supervisor/user). If you know Cortex-M, RISC-V is mostly relabeling.

---

15. C-language gotchas every embedded interview probes

- `volatile` — tells the compiler the value can change outside program flow (hardware register, ISR-modified flag). Missing `volatile` on a register or a flag set in an ISR → compiler caches it in a register and your `while(!flag);` loops forever at -O2. **Top-3 embedded bug.**
- `volatile` **is not atomic** — a `volatile uint32_t` shared with an ISR still needs a critical section for read-modify-write, and on a 32-bit core even a plain 64-bit access isn't atomic.
- **Alignment** — Cortex-M mostly allows unaligned word access *except* for some instructions and except when the trap is enabled; casting a `uint8_t*` buffer to a `uint32_t*` can fault. Relevant to packing/unpacking protocol frames (your ADC frames).
- **Bitfields** — implementation-defined ordering/padding; do **not** map a struct bitfield onto a hardware register and assume bit positions. Use explicit masks/shifts.
- `static` for file-scope encapsulation and persistent locals; `const` can place data in Flash (saves RAM).
- `int` **size** — on Cortex-M `int` is 32-bit; don't assume 16-bit habits from 8-bit MCUs.

---

16. Rapid-fire interview Q&A (cover and recall)

1. **First two words of the vector table?** Initial MSP, then reset vector address.
2. **Where does the core start executing and in what mode?** Reset_Handler, Thread mode, privileged, MSP.
3. **What gets auto-stacked on an interrupt?** R0–R3, R12, LR, PC, xPSR (+ FPU regs if lazy-stacking triggers).
4. **Why is the first stacked PC useful?** It's the return/fault address — pinpoints a crash.
5. **Difference between preemption and sub-priority?** Preemption causes nesting; sub-priority only orders equal-preemption pending IRQs.
6. **How does an RTOS separate task stacks?** Tasks run on PSP, kernel/ISRs on MSP; PendSV switches the PSP frame.
7. **Why might a peripheral be completely dead?** RCC clock-enable bit not set.
8. **What's the DMA+cache hazard on M7?** Clean D-cache before TX, invalidate after RX, 32-byte align — or use DTCM/non-cacheable region.
9. **Why won't my `while(!flag)` exit?** `flag` not declared `volatile`; compiler hoisted the read.
10. **MPU vs MMU?** MPU = access permissions on fixed regions, no virtual memory; MMU = translation + paging (Cortex-A, not M).
11. **What is EXC_RETURN?** Magic LR value on exception entry encoding return mode/stack; loading it into PC triggers unstacking.
12. **Bare-metal: what must startup do before main?** Set SP (HW does word0), copy `.data`, zero `.bss`, run C++ ctors, optionally init clocks.

---

17. Things to turn into daily code snippets

- Bare-metal blink: enable GPIO clock in RCC, set MODER, write ODR — **no HAL**.
- Read `SCB->CPUID` and decode the part/revision.
- Minimal HardFault handler that captures the stacked frame (§6) and breakpoints.
- SysTick at 1 kHz by hand; a `delay_ms` using it.
- Configure one NVIC priority with the 4-bit shift and prove preemption with two IRQs.
- Place a buffer in DTCM (linker section) and DMA into it; compare against a cached-region buffer to *see* the coherency bug, then fix it with invalidate.

Next: feed me a chapter (Yiu's "Definitive Guide to the Cortex-M3/M4/M7" or the STM32F7 Reference Manual / Programming Manual PM0253) and I'll deepen any section above with exact register bitfields and worked examples.

Part 2 — GPIO & External Interrupts (EXTI)

Microcontrollers — GPIO & External Interrupts (EXTI)

> Question-first notes. STM32F746 register names used as the concrete example; the *concepts* (modes, push-pull vs open-drain, atomic set/reset, edge interrupts) are universal.

1. The GPIO register model

Q: What registers configure one STM32 GPIO port, and what does each do?

Each port (GPIOA, GPIOB…) is a struct of 32-bit registers:

Register	Width/pin	Purpose
`MODER`	2 bits	Mode: 00 input, 01 output, 10 alternate function, 11 analog
`OTYPER`	1 bit	Output type: 0 push-pull, 1 open-drain
`OSPEEDR`	2 bits	Output speed / slew rate (low→very-high)
`PUPDR`	2 bits	Pull: 00 none, 01 pull-up, 10 pull-down
`IDR`	1 bit (r)	Input data — read pin state
`ODR`	1 bit	Output data — read/write (but **don't** RMW it, see §4)
`BSRR`	set/reset	**Atomic** set (low 16) / reset (high 16) of output bits
`AFRL/AFRH`	4 bits	Alternate-function selector per pin (AF0–AF15)
`LCKR`	—	Lock a pin's config until reset

Gotcha #0: the port does nothing until its clock is enabled: `RCC->AHB1ENR |= RCC_AHB1ENR_GPIOBEN;` — then a dummy read is good practice before configuring.

2. The four pin modes

Q: What are the GPIO modes and when do you use each?

- **Input** — read a pin (button, sensor digital line). Pair with pull-up/down.
- **Output** — drive a pin (LED, chip-select, enable).
- **Alternate Function (AF)** — hand the pin to a peripheral (USART TX, SPI SCK, TIM PWM). You set `MODER=10` *and* pick the AF number in `AFRL/AFRH`.
- **Analog** — disconnects the digital input buffer; required for ADC/DAC pins and lowest-power unused pins.

Gotcha: for a peripheral pin, setting AF mode is not enough — you must also program the **correct AF number**. "My UART TX is silent" is often `MODER` set to AF but `AFRL` left at AF0. The AF mapping table is in the datasheet (not the reference manual) — pin PA9 → USART1_TX is AF7 on the F746, for instance.

3. Push-pull vs open-drain, pull resistors, speed

Q: Push-pull vs open-drain — when does it matter?

- **Push-pull** — actively drives both high and low. Default for LEDs, CS, most outputs.
- **Open-drain** — can only pull low; high is "released" (high-Z), needs an external pull-up. Required for **shared/wired-AND buses**: I2C SDA/SCL, or any line multiple devices may pull low.

Gotcha (I2C): if you configure I2C pins as push-pull, two devices driving opposite levels fight and you get bus contention / stuck-low SDA. I2C pins **must** be open-drain with pull-ups (internal PUPDR pull-ups are usually too weak — use external 2.2k–4.7k).

Q: Output speed / slew rate — free performance?

No — higher speed = faster edges = more **EMI and ringing**. Use the *lowest* speed that meets your signal's timing. Cranking everything to very-high is a classic source of overshoot and radiated noise. High-speed SPI/clock pins need it; an LED does not.

---

4. The atomic set/reset gotcha (BSRR)

Q: Why use BSRR instead of writing ODR?

Writing a single bit of `ODR` is a **read-modify-write**: read all 16 bits, change one, write back. If an interrupt fires mid-sequence and also touches that port, you get a lost update / race.

`BSRR` is **write-only and atomic**: low 16 bits set pins, high 16 bits reset them, in one bus write, no read. Always toggle pins via BSRR when an ISR might touch the same port.

```
GPIOB->BSRR = (1u << 12);          // set PB12 (e.g. CS high) — atomic
GPIOB->BSRR = (1u << (12 + 16));   // reset PB12 (CS low)     — atomic
```

Direct tie to your work: your ADS131M08 chip-select edges between SPI frames are exactly the kind of pin you want driven via BSRR, especially with DMA/interrupts in flight.

---

5. EXTI — external interrupts

Q: How does an STM32 turn a pin edge into an interrupt?

1. **SYSCFG** maps a pin to an EXTI line: `SYSCFG->EXTICR[n]` selects which *port* drives EXTI line `x` (EXTI line number = pin number, so PB0 and PA0 both compete for EXTI0 — only one port per line).
2. **EXTI** config: set edge in `EXTI->RTSR` (rising) / `EXTI->FTSR` (falling), unmask in `EXTI->IMR` (interrupt) or `EXTI->EMR` (event-only, no CPU interrupt).
3. **NVIC**: enable the corresponding IRQ.
4. In the ISR: check & **clear the pending flag** by writing 1 to `EXTI->PR` (write-1-to-clear).

Q: Interrupt vs event?

- **Interrupt** (IMR) → CPU vectors to an ISR.
- **Event** (EMR) → generates a hardware pulse/wakeup *without* running code (used to wake from low-power WFE or trigger another peripheral). No ISR runs.

Gotcha #1: forgetting to enable the **SYSCFG clock** (`RCC->APB2ENR |= RCC_APB2ENR_SYSCFGEN`) before writing EXTICR → the pin→line mapping silently doesn't take.

Gotcha #2: shared vectors. Lines 5–9 share one NVIC vector (`EXTI9_5_IRQn`), lines 10–15 share another (`EXTI15_10_IRQn`). One ISR must check `EXTI->PR` to find *which* line fired and clear the right bit. Handling only one line in a shared ISR = missed events.

Gotcha #3: not clearing `EXTI->PR` → the ISR re-fires forever (interrupt storm). Write-1-to-clear.

Gotcha #4: one EXTI line can only watch one port at a time. You cannot have both PA0 and PB0 as separate EXTI0 interrupts simultaneously.

---

6. Debouncing

Q: A mechanical button generates multiple edges — how do you handle it?

- **Hardware:** RC filter + Schmitt-trigger input, or an RC across the switch.

- **Software:** on the EXTI interrupt, start a short timer (e.g. 10–20 ms) and only accept the state if it's stable when the timer expires; or sample the pin periodically and require N consecutive equal reads.
- **Common pattern:** EXTI fires → disable that EXTI line → start debounce timer → on timeout, read final state and re-enable the line.

Gotcha: debouncing purely inside the EXTI ISR with a blocking delay is wrong — it blocks the CPU and can still miss bounces. Use a timer or periodic sampling.

---

7. Minimal bare-metal blink (no HAL)

```
// PB7 as push-pull output, toggle. F746 Discovery-style.
RCC->AHB1ENR |= RCC_AHB1ENR_GPIOBEN;     // 1. clock the port
(void)RCC->AHB1ENR;                       //    dummy read
GPIOB->MODER  &= ~(3u << (7 * 2));        // 2. clear mode bits
GPIOB->MODER  |=  (1u << (7 * 2));        //    01 = output
GPIOB->OTYPER &= ~(1u << 7);              //    push-pull
GPIOB->OSPEEDR &= ~(3u << (7 * 2));       //    low speed (an LED)

for (;;) {
    GPIOB->BSRR = (1u << 7);              // set   (atomic)
    for (volatile int i = 0; i < 800000; i++);
    GPIOB->BSRR = (1u << (7 + 16));       // reset (atomic)
    for (volatile int i = 0; i < 800000; i++);
}
```

This snippet *is* your "do I understand GPIO" test — type it from memory and flash it.

---

8. Interview rapid-fire

1. **Which register gives atomic pin set/reset, and why prefer it?** BSRR — write-only, no RMW race with ISRs.
2. **Why must I2C pins be open-drain?** Wired-AND shared bus; push-pull would cause contention.
3. **What two things must you set for a peripheral pin?** MODER=AF *and* the AF number in AFRL/AFRH.
4. **What does analog mode do?** Disconnects the digital input buffer (for ADC pins / low-power unused pins).
5. **Why might EXTI not fire at all?** SYSCFG clock not enabled, or EXTICR not pointing at the right port.
6. **Why does my EXTI ISR fire endlessly?** Pending flag (`EXTI->PR`) not cleared (write-1-to-clear).
7. **Can PA0 and PB0 both be EXTI interrupts at once?** No — one port per EXTI line.
8. **Tradeoff of high GPIO output speed?** Faster edges → more EMI/ringing; use the slowest that works.
9. **Difference between EXTI interrupt and event mode?** Interrupt runs an ISR; event just pulses hardware / wakes from WFE, no code.

Deepen later from the STM32F7 reference manual (RM0385/RM0410) GPIO + SYSCFG + EXTI chapters, and the F746 datasheet's alternate-function mapping table.

Part 3 — Timers, PWM, Watchdog & RTC

Microcontrollers — Timers, PWM, Watchdogs & RTC

> Question-first notes. STM32F746 as the concrete example. Timers are one of the most heavily interviewed peripherals and the most useful in practice (PWM, precise timing, measurement, motor control).

---

1. Timer families on STM32

Q: What classes of timer exist and how do they differ?

- **Basic** (TIM6, TIM7) — just a counter + update event. Used for time-base / triggering DAC/DMA. No input/output channels.
- **General-purpose** (TIM2–5, TIM9–14…) — up/down/center counting, multiple capture/compare channels → PWM, input capture, output compare, encoder mode.
- **Advanced** (TIM1, TIM8) — everything general-purpose has **plus** complementary outputs, **dead-time insertion**, and **break input**. Built for motor / power control (3-phase bridges).

Gotcha: TIM2 and TIM5 are **32-bit** counters on STM32F4/F7; most others are 16-bit. If you need a long free-running counter (e.g. microsecond timestamping), reach for a 32-bit timer to avoid frequent overflow handling.

---

2. The core counting mechanism

Q: How do PSC and ARR set the timer frequency?

- **PSC** (prescaler) divides the timer clock: `counter_clock = timer_clk / (PSC + 1)`.
- **CNT** counts up to **ARR** (auto-reload), then generates an **Update Event (UEV)** and reloads.
- Output/update frequency: `f_update = timer_clk / ((PSC + 1) * (ARR + 1))`.

Gotcha #1 — the N−1 off-by-one: both PSC and ARR are "value − 1." For a 1 kHz update from a 1 MHz counter clock you write `ARR = 999`, not 1000. Forgetting the −1 is the single most common timer bug.

Gotcha #2 — the 2× timer clock: on STM32 the timer's input clock is often **2× the APBx peripheral clock** when the APB prescaler ≠ 1. So a timer on a 54 MHz APB1 may actually clock at 108 MHz. Get this wrong and every PWM frequency / delay is off by 2×. (Cross-reference the clock- tree note.)

Q: Counting modes?

- **Up** — 0→ARR→0.
- **Down** — ARR→0→ARR.
- **Center-aligned** — up then down; used for symmetric PWM (lower harmonics in motor drives).

---

3. ARR/CCR preload & shadow registers

Q: What is the preload (shadow) mechanism and why does it matter?

ARR and the compare registers (CCR) can be **buffered**: you write a new value, but it only takes effect at the next update event, not mid-cycle. Enabled via `ARPE` (ARR preload) and `OCxPE` (CCR preload).

Gotcha: changing PWM duty on the fly **without** preload can produce a glitched/runt pulse because the compare value changes mid-period. Enable preload for clean transitions. Conversely, if you *want* an immediate change you must disable preload — know which you've set.

---

4. PWM generation

Q: How do you generate PWM on a timer channel?

1. Set PSC/ARR for the desired **frequency**.
2. Put the channel in **PWM mode** (`CCMRx` OCxM bits = PWM mode 1 or 2).
3. Set **CCRx** for the **duty** (duty = CCRx / (ARR+1)).
4. Enable channel output `CCER` (CCxE) and set polarity.
5. **For advanced timers (TIM1/8): set `BDTR`'s MOE (Main Output Enable) bit** or there is no output at all.
6. Enable the counter (`CR1` CEN).

Gotcha (advanced-timer "PWM is dead"): TIM1/TIM8 outputs are gated by **MOE**. People configure everything correctly, see the counter running, but get nothing on the pin because MOE is off (or a break event cleared it). General-purpose timers don't need MOE — only advanced ones.

```
// 20 kHz PWM, 25% duty on TIM3_CH1, timer_clk = 108 MHz (example)
TIM3->PSC  = 0;                     // counter_clk = 108 MHz
TIM3->ARR  = 5399;                  // 108e6/5400 = 20 kHz  (note the -1)
TIM3->CCR1 = 1350;                  // 1350/5400 = 25% duty
TIM3->CCMR1 |= (6 << 4);            // OC1M = PWM mode 1
TIM3->CCMR1 |= TIM_CCMR1_OC1PE;     // CCR1 preload
TIM3->CCER  |= TIM_CCER_CC1E;       // enable CH1 output
TIM3->CR1   |= TIM_CR1_ARPE;        // ARR preload
TIM3->CR1   |= TIM_CR1_CEN;         // start
// (TIM3 is general-purpose: no MOE needed)
```

---

5. Output compare & input capture

Q: Output compare vs input capture?

- **Output compare (OC):** when CNT == CCR, do something to the output pin (set/clear/toggle) or fire an interrupt/DMA. PWM is a special case of OC.
- **Input capture (IC):** an edge on the input pin **latches CNT into CCR** and can interrupt. This is how you *measure* external signals.

Q: How do you measure a signal's frequency / pulse width with input capture?

- **Frequency / period:** capture on rising edges, period = `CCR_now − CCR_prev` (× counter tick), handle counter wrap.
- **Pulse width / duty (PWM input mode):** two channels capture the same input — one on rising, one on falling — giving period and high-time together in hardware.

Gotcha: if the captured value can exceed one counter period, you must count overflow (UEV) events too, or your measurement aliases. Pick PSC so the expected period fits comfortably below ARR.

---

6. Encoder & one-pulse modes

Q: What is encoder interface mode?

The timer decodes a **quadrature encoder** (two phase signals A/B) directly in hardware: CNT increments/decrements with direction, no CPU per-edge. Read CNT to get position. Great for motors and dials — zero software overhead.

One-pulse mode (OPM): the timer runs exactly one cycle after a trigger then stops — generates a single precise delayed pulse (e.g. a trigger-to-action delay).

---

7. Advanced-timer extras (motor/power control)

Q: What do complementary outputs, dead-time, and break input give you?

- **Complementary outputs** (CHx and CHxN) — drive the high-side and low-side of a half-bridge with inverted signals.
- **Dead-time insertion** — a programmable gap where *both* outputs are off during the transition, so the two MOSFETs of a leg are never on together (shoot-through protection). Set in `BDTR`.
- **Break input** — an external fault pin (overcurrent, fault) instantly forces outputs to a safe state in hardware, faster than any ISR.

These are why TIM1/TIM8 exist; if you ever interview for motor control, this section is the meat.

---

8. Timer + DMA

Q: How do timers drive DMA?

Timer events (update, capture/compare) can raise **DMA requests** — e.g. feed a buffer of values to GPIO/DAC at a fixed rate, or capture a stream of edges into RAM without CPU per-sample. **DMA burst mode** lets one trigger write several timer registers at once. This is how you get jitter-free, CPU-free periodic I/O — relevant to deterministic DAQ sampling.

---

9. Watchdogs

Q: IWDG vs WWDG — what's the difference and why use a watchdog?

A watchdog resets the MCU if software fails to "kick" it in time, recovering from hangs/lockups.

- **IWDG (Independent Watchdog):** clocked by the independent **LSI** RC oscillator, so it keeps running even if the main clock dies. Simple down-counter; refresh before timeout. Best for catching total lockups.
- **WWDG (Window Watchdog):** clocked from APB; you must refresh inside a **window** — not too late **and not too early**. Refreshing too soon also triggers reset. Catches code that's running too fast / out of its expected timing, not just hung code.

Gotcha #1: kicking the watchdog from a *timer interrupt* defeats the purpose — if `main()` hangs but the ISR still runs, the dog never barks. Kick from the main loop / supervised task.

Gotcha #2: IWDG timeout depends on the LSI frequency, which is imprecise (±several %). Don't design a tight margin; leave slack.

---

10. RTC (Real-Time Clock)

Q: What does the RTC provide and what's special about it?

Calendar (date/time with BCD registers), **alarms**, periodic **wakeup** (including from low-power Stop/Standby), timestamp, and tamper detection. It lives in the **backup domain**, powered by **VBAT** (coin cell), so it keeps time through main-power loss.

Gotcha #1: the backup domain is **write-protected** by default — you must set `PWR` DBP bit to access RTC/backup registers, and follow the init sequence (disable write protection, enter init mode, set prescalers, exit).

Gotcha #2: RTC registers are clocked slowly (32.768 kHz LSE); after writing you often must wait for synchronization (`RSF`) before reads are valid.

Gotcha #3: without an external **LSE crystal** the RTC falls back to imprecise LSI — clocks drift noticeably. Use a 32.768 kHz crystal for real timekeeping.

---

11. Interview rapid-fire

1. **f_update formula?** `timer_clk / ((PSC+1)*(ARR+1))`.
2. **Why ARR = 999 for 1000 counts?** Registers are value−1.
3. **Why is my PWM frequency 2× off?** Timer clock is often 2× APB clock when APB prescaler ≠ 1.
4. **Advanced-timer PWM gives no output — why?** MOE bit in BDTR not set.
5. **What does preload/ARPE prevent?** Glitched/runt pulses when changing duty mid-period.
6. **How to measure an external frequency in hardware?** Input capture; period = CCR_now − CCR_prev.
7. **IWDG vs WWDG?** IWDG = independent LSI clock, simple timeout (catches hangs); WWDG = APB-clocked window (catches too-fast/too-slow).
8. **Why not kick the watchdog from an ISR?** A hung main loop with a live ISR would never reset.
9. **What is dead-time and why?** Gap with both half-bridge outputs off, prevents shoot-through.
10. **Which timers are 32-bit on F4/F7?** TIM2 and TIM5.
11. **What keeps the RTC running with main power off?** Backup domain on VBAT, LSE/LSI clock.

---

12. Daily-snippet candidates

- 20 kHz PWM with runtime duty change using CCR preload (§4).
- Input capture measuring an external square wave's frequency, wrap-handled.
- IWDG setup + a deliberate hang to watch it reset; then fix the kick.
- RTC: set time, set an alarm, wake from Stop mode on the alarm.
- Encoder mode reading a quadrature signal into CNT.

Deepen later from the STM32F7 reference manual timer chapters (general-purpose vs advanced-control timer), plus the IWDG/WWDG and RTC chapters.

Part 4 — ADC & DAC

Microcontrollers — ADC & DAC

> Question-first notes. STM32F746 internal **SAR** ADC as the concrete example, contrasted with your external **ADS131M08 sigma-delta** ADC where it's illuminating.

1. How a SAR ADC works

Q: Explain successive-approximation conversion.

A sample-and-hold capacitor grabs the input voltage, then the ADC binary-searches the value: it guesses the MSB (is Vin > Vref/2?), keeps or drops it, then the next bit, and so on — N comparisons for N bits, via an internal DAC + comparator. Output = `Vin / Vref × (2^N − 1)`.

- **Resolution:** STM32 SAR is typically **12-bit** (also 6/8/10 selectable). LSB = `Vref / 2^N`. At Vref = 3.3 V, 12-bit ⇒ 1 LSB ≈ 0.806 mV.
- **Speed:** fast (MSPS range), one conversion = sampling time + ~12 ADC clock cycles.

Q: SAR vs sigma-delta — the contrast that matters for you.

	SAR (STM32 internal)	Sigma-delta (your ADS131M08)
Method	Binary search, one shot	Oversample + noise-shape + decimate
Resolution	12–16 bit	24-bit, very high
Speed	Fast (MSPS)	Slower per sample (kSPS), but many channels simultaneously
Noise	Moderate	Excellent (noise pushed out of band)
Latency	Low, immediate	Has group delay from the digital filter
Use	General sensing, fast multi-ch scan	Precision measurement, DAQ, your ADC

The key intuition: SAR trades resolution for speed and immediacy; sigma-delta trades latency for resolution by oversampling far above Nyquist and filtering. That filter group delay is why your ADS131M08 data has a settling/latency characteristic the STM32 SAR doesn't.

2. Sampling time — the #1 ADC gotcha

Q: Why does sampling time matter, and what goes wrong if it's too short?

The sample-and-hold cap must charge to the input voltage *through the source impedance* before conversion. If sampling time is too short for a high-impedance source, the cap never fully charges ⇒ the reading is **low and noisy**. Required sampling time ∝ source impedance.

Gotcha: "my ADC reading is wrong / drifts with source" → almost always **sampling time too short for the source impedance**. High-impedance sensors (some thermistors, dividers) need long sample times or an op-amp buffer in front. The reference manual gives a max source impedance per sampling-time setting — respect it.

3. Conversion modes

Q: Single vs continuous vs scan vs discontinuous; regular vs injected?

- **Single** — one conversion, then stop.
- **Continuous** — re-triggers itself forever.

- **Scan** — convert a *sequence* of channels automatically (multi-channel).
- **Discontinuous** — convert the sequence in small sub-groups per trigger.
- **Regular group** — the normal sequence.
- **Injected group** — higher-priority conversions that *interrupt* the regular sequence (like an interrupt for the ADC) — used for urgent measurements (e.g. motor current) mid-scan.

Q: How do you get a precise, jitter-free sample rate?

Don't trigger conversions in software/timer ISR (jitter). Use a **timer TRGO** to hardware-trigger the ADC at an exact rate, with **DMA** moving results out. This is the deterministic-sampling pattern and the right answer in interviews.

---

4. ADC + DMA (the pattern you'll use constantly)

Q: Why pair the ADC with DMA, and how?

In continuous/scan mode the ADC produces a result every conversion; without DMA you'd need an ISR per sample (impossible at high rates). DMA in **circular mode** copies each result into a RAM buffer automatically; you process on **half-transfer** and **transfer-complete** interrupts (double-buffer style — process the half the DMA isn't currently writing).

```
// sketch: timer-triggered, DMA circular, multi-channel scan
// 1. ADC: scan mode, ext trigger = TIMx TRGO, DMA enabled, continuous DMA requests
// 2. TIM:  TRGO on update at the sample rate
// 3. DMA:  peripheral=ADC->DR, memory=buf[], circular, half+full IRQ
// 4. In HT IRQ: process buf[0 .. N/2);  in TC IRQ: process buf[N/2 .. N)
```

Gotcha (F7 cache + DMA): the ADC DMA buffer is written by DMA but read by the CPU through the D-cache — you must **invalidate the cache** for that buffer before reading, 32-byte aligned. And on F7 the DMA controllers **can't reach DTCM** — see the Memory/DMA notes; put ADC DMA buffers in SRAM, not DTCM. (This refines the "use DTCM" tip from the core file: DTCM dodges cache but is DMA-invisible on F7.)

---

5. Resolution, Vref, calibration

Q: What sets accuracy beyond the bit count?

- **Vref accuracy** — the whole scale is relative to Vref; a noisy/inaccurate Vref ruins absolute accuracy regardless of resolution. Use a clean reference for measurements that matter.
- **Calibration** — STM32 ADCs have a self-calibration routine (offset) you run at startup; skipping it leaves an offset error.
- **VREFINT internal channel** — a known internal voltage you can measure to *compute the actual Vref/VDDA* at runtime (useful on battery where VDDA sags).

Internal channels: temperature sensor, VBAT/n monitor, VREFINT. The temp sensor needs the datasheet calibration constants (factory values in system memory) to convert to °C — don't use the raw formula.

---

6. Aliasing & oversampling

Q: What is aliasing and how do you avoid it?

Per Nyquist, sample at > 2× the highest signal frequency. Frequencies above Fs/2 **fold back** into your band as fake low-frequency content. Fix: an **analog anti-alias low-pass filter** before the ADC. (A digital filter after sampling can't undo aliasing — the damage is done at sample time.)

Q: Oversampling for more effective bits?

Sampling faster than needed and averaging reduces noise: averaging 4^n samples gains ~n bits of *effective* resolution. STM32F7 has a **hardware oversampler** that does this in the ADC and gives you a wider result directly. (This is, in spirit, a slice of what sigma-delta does continuously.)

---

7. DAC

Q: How does the STM32 DAC work and what are its modes?

A true voltage-output DAC (R-2R style), typically 12-bit, 1–2 channels. Write `DHRx`, output appears on the pin. Features:

- **Output buffer** — drives loads but limits range near the rails; can be disabled for full-rail swing into high-impedance loads.
- **Triggering + DMA** — feed a buffer of samples via DMA on a **timer trigger** to synthesize arbitrary waveforms at a fixed rate (DDS-style: a sine lookup table → smooth sine out).
- **Built-in noise / triangle generators** — hardware LFSR noise and triangle wave without a table.

```
// arbitrary waveform: TIM TRGO -> DAC -> DMA from sine_lut[]
// sample_rate = TRGO rate; output freq = sample_rate / LUT_length
```

Gotcha: the output buffer can't reach the last few tens of mV near GND/VDDA. For full-scale swing into high-Z, disable the buffer (at the cost of drive strength).

---

8. Practical / layout gotchas

- **Pin in analog mode** — the ADC/DAC pin's `MODER` must be `11` (analog), or you read garbage.
- **VDDA/VSSA** — analog supply must be properly decoupled and tied; noisy analog ground wrecks LSBs.
- **Reading mid-conversion** — read `DR` only after EOC, or via DMA; polling the wrong flag gives stale/partial data.
- **Common-mode / grounding** for your external sigma-delta DAQ matters even more — single-point ground, keep digital switching noise off the analog return.

---

9. Interview rapid-fire

1. **How many comparisons for an N-bit SAR conversion?** N.
2. **LSB voltage formula?** Vref / 2^N.
3. **Reading is low/noisy with a high-impedance sensor — why?** Sampling time too short to charge the S/H cap.
4. **How to sample at an exact rate?** Timer TRGO hardware-triggers the ADC + DMA out (not software).
5. **Regular vs injected group?** Injected preempts regular — high-priority urgent conversions.
6. **SAR vs sigma-delta in one line?** SAR = fast binary search, moderate bits; sigma-delta = oversample+noise-shape, high bits, has latency.
7. **Why an analog anti-alias filter, not digital?** Aliasing happens at sample time; digital can't recover folded frequencies.
8. **How do you get extra effective bits?** Oversample + average (or the HW oversampler).
9. **Why measure VREFINT?** To compute actual VDDA/Vref at runtime (battery sag).
10. **F7 ADC DMA buffer — two hazards?** D-cache (invalidate, 32-byte align) and DTCM not DMA-accessible (use SRAM).

---

10. Daily-snippet candidates

- Single-channel polled read, then convert to millivolts with measured VREFINT.
- Timer-triggered + DMA circular multi-channel scan; process on HT/TC with cache invalidate.
- Read the internal temperature sensor using factory calibration constants.
- DAC sine output via timer-triggered DMA from a LUT; change frequency by changing TRGO rate.

Deepen later from the STM32F7 reference manual ADC + DAC chapters, and AN2834 (getting the best ADC accuracy) / AN3116. For your DAQ, the ADS131M08 datasheet's digital-filter/group-delay section is the sigma-delta counterpart.

Part 5 — Memory, DMA & Flash

Microcontrollers — Memory, DMA & Flash

> Question-first notes. STM32F746 memory architecture as the concrete example. This file ties together where data lives, how it moves without the CPU (DMA), and how non-volatile storage (Flash) behaves.

1. The memory landscape on a modern MCU

Q: What memory types are on an STM32F7 and how do they differ?

Memory	Volatile?	Speed	Cached?	DMA-accessible?	Use
Flash	No	Slow (wait states)	via ART/prefetch	read by DMA	code + constants
DTCM-RAM	Yes	Single-cycle, fastest	**No** (never cached)	**No (F7!)**	stacks, hot CPU data
ITCM-RAM	Yes	Single-cycle	No	No	hot code
SRAM1/SRAM2	Yes	Fast (AHB)	**Yes (D-cache)**	**Yes**	general data, DMA buffers
Backup SRAM	Yes (VBAT)	Slow	No	—	retained data across reset
External (FMC/QSPI)	varies	slower	configurable	yes	big buffers, framebuffers

The single most important F7 gotcha lives in this table — see §4.

2. Flash internals

Q: What's special about reading, programming, and erasing Flash?

- **Read** is normal memory access, but Flash is slower than the core, so you set **wait states** (latency) in `FLASH->ACR` matching the clock/voltage. The F7 also has the **ART Accelerator** (prefetch + instruction cache) to hide latency.
- **Program** is done in units (byte/half-word/word/double-word depending on config); you can only flip bits **1→0** when programming.
- **Erase** sets a whole **sector** back to all-1s (0xFF). You cannot erase a single byte — erase is sector-granular. Sectors on F7 are *non-uniform* (some 16 KB, some 64 KB, some 256 KB).
- **Endurance** is limited (~10k+ erase cycles) — don't treat Flash like RAM for frequent writes (use external EEPROM/FRAM or wear-leveling for logging).

Q: How do you program Flash from firmware (IAP / bootloader)?

Unlock with the **key sequence** to `FLASH->KEYR`, set the program-size bits, write, poll BSY, then re-lock. **Gotcha:** you generally **cannot execute or read from the same Flash bank you're erasing** — the bus stalls. Run the programming routine from RAM (or the other bank on dual-bank parts), which is exactly what bootloaders do.

Q: Wait-state ordering gotcha?

When **raising** the clock you must **increase wait states first, then switch the clock up**; when **lowering**, drop the clock first, then reduce wait states. Doing it in the wrong order means the CPU fetches from Flash faster than it can respond ⇒ corruption / HardFault. This is a classic clock-setup bug.

Other Flash items: option bytes (boot config, BOR level, watchdog mode), **RDP read protection** (levels 0/1/2 — level 2 is irreversible and locks debug), **ECC** on F7 Flash (detects/ corrects bit errors).

---

3. Linker script & memory sections

Q: What does the linker script control, and what are the sections?

It defines memory **regions** (FLASH, DTCM, SRAM…) and places **sections** into them:

- `.text` — code (Flash)
- `.rodata` — const data (Flash)
- `.data` — initialized globals (lives in Flash, **copied to RAM** at startup)
- `.bss` — zero-initialized globals (RAM, **zeroed** at startup)
- `heap` / `stack` — placed in RAM (often DTCM for speed)

Q: How do you put a buffer in a specific RAM?

Define a section in the linker script mapped to that region, then attribute the variable:

```
// place a fast scratch buffer in DTCM (CPU-only, not DMA!)
uint32_t hot_buf[256] __attribute__((section(".dtcm")));

// place a DMA buffer in SRAM1, cache-line aligned
uint8_t  dma_buf[512] __attribute__((section(".sram1"), aligned(32)));
```

Gotcha: mismatched region sizes/placement → linker "region overflowed" or, worse, a buffer silently in the wrong RAM (see DMA below). Always check the `.map` file for where things actually landed.

---

4. DMA — the heart of CPU-free data movement

Q: What is DMA and why is it essential?

A DMA controller is a bus master that moves data between peripheral↔memory or memory↔memory **without the CPU**, freeing it during transfers and enabling rates the CPU couldn't service per-byte. Essential for your DAQ: SPI/ADC streams land in RAM automatically.

Q: Streams, channels, request mapping?

- STM32F7 has **DMA1/DMA2**, each with **streams**; each stream can serve one of several mapped **channels** (peripheral requests). The peripheral→stream/channel mapping is **fixed by a table** on F7 (newer STM32 use a flexible **DMAMUX**). You must pick the stream/channel the datasheet assigns to your peripheral — choosing the wrong one means no transfer.
- **Priorities** arbitrate between active streams.

Q: DMA transfer modes?

- **Normal** — transfer N items then stop.
- **Circular** — wraps to the start automatically (continuous ADC/SPI streaming).
- **Double-buffer** — ping-pong between two buffers; CPU processes one while DMA fills the other.

Q: Increment, width, FIFO, burst?

- **Address increment** — increment memory pointer (fill a buffer), keep peripheral pointer fixed (a data register).
- **Data width** — byte/half-word/word; peripheral and memory widths can differ (packing).
- **FIFO + burst** — buffers data so DMA can do efficient burst transfers on the bus; FIFO threshold must be consistent with the burst/width or you get a config error.

Q: DMA interrupts?

Half-transfer (HT), transfer-complete (TC), transfer-error (TE), FIFO error. HT+TC together give the double-buffer-in-one-circular-buffer pattern.

4a. The big F7 DMA gotchas (these bite real projects)

- **DMA cannot access DTCM on F7.** DTCM is behind the CPU's TCM interface, not on the AHB bus matrix the DMA controllers live on. A DMA buffer placed in DTCM ⇒ transfer fails/zeros, often silently. **DMA buffers go in SRAM1/SRAM2** (or external RAM), never DTCM. This refines the core- file tip: DTCM is great for *CPU-only* hot data to dodge cache, but it's invisible to DMA.
- **D-cache coherency** (cross-ref core file §9): SRAM is cached, DMA is not. Before a memory→peripheral DMA, **clean** the cache; after a peripheral→memory DMA, **invalidate** before reading. Buffers must be **32-byte aligned** and length rounded to the cache line, or you corrupt neighbors.
- Clean alternative: mark the DMA buffer's SRAM region **non-cacheable via the MPU** and skip manual clean/invalidate entirely — common in DAQ designs.

---

5. Putting it together (your DAQ shape)

For 8 kSPS streaming from the ADS131M08 over SPI+DMA: 1. DMA buffer in **SRAM** (not DTCM), 32-byte aligned, circular or double-buffered. 2. SPI RX DMA fills it continuously; HT/TC interrupts hand half-buffers to processing. 3. **Invalidate D-cache** for the half being read, or put the buffer in an MPU non-cacheable region. 4. Keep per-task stacks in **DTCM** for speed (CPU-only — fine there).

---

6. Interview rapid-fire

1. **Erase granularity of Flash?** A whole sector → 0xFF; can't erase one byte.
2. **Which bit direction does programming allow?** 1→0 only; erase restores 1s.
3. **Wait-state ordering when changing clock?** Raise WS before raising clock; lower clock before lowering WS.
4. **Why run Flash-programming code from RAM?** Can't fetch from the bank being erased.
5. `.data` **vs** `.bss` **at startup?** `.data` copied from Flash to RAM; `.bss` zeroed.
6. **What is DMA circular mode for?** Continuous streaming (ADC/SPI) with auto-wrap.
7. **What do HT + TC interrupts enable?** Double-buffer processing within one circular buffer.
8. **Why won't my DMA into DTCM work on F7?** DMA controllers can't access DTCM — use SRAM.
9. **Two cache rules for DMA on M7?** Clean before TX, invalidate after RX (32-byte aligned).
10. **What hides Flash latency on F7?** ART accelerator (prefetch + instruction cache) + wait states.
11. **Why not log frequently to internal Flash?** Limited erase endurance + sector-granular erase.

---

7. Daily-snippet candidates

- Memory-to-memory DMA copy; compare speed vs `memcpy`.
- SPI/ADC peripheral→memory DMA in circular mode with HT/TC processing + cache invalidate.
- Place one buffer in DTCM and one in SRAM; DMA into both and observe DTCM fails on F7.
- Bare-metal Flash erase+program of one sector (unlock sequence), routine run from RAM.
- Read the `.map` file and confirm where `.data`, `.bss`, stack, and your buffers landed.

Deepen later from the STM32F7 reference manual Flash + DMA chapters, the programming manual for the linker/startup, and AN4839 (STM32F7 caches / DMA coherency).

Part 6 — Clock & Power Deep-Dive

Microcontrollers — Clock & Power Deep-Dive

> Question-first notes. STM32F746 (216 MHz, over-drive) as the concrete example. This is where "the chip won't run fast / won't wake / browns out / peripheral is dead" bugs come from.

1. Clock sources

Q: What oscillators does an STM32F7 have?

- **HSI** — High-Speed Internal RC, ~16 MHz. Starts instantly, no external parts, but imprecise (±1–2 %, temperature-dependent). Default clock at reset.
- **HSE** — High-Speed External, a crystal (e.g. 8/25 MHz) or external clock. Precise; required for reliable USB/Ethernet/CAN timing.
- **LSI** — Low-Speed Internal RC, ~32 kHz. Imprecise; clocks IWDG and can clock the RTC as fallback.
- **LSE** — Low-Speed External, 32.768 kHz crystal. Precise; the proper RTC clock.
- **PLL(s)** — multiply a source (HSI or HSE) up to the system clock and peripheral clocks.

Gotcha: every source has a **ready flag** (`HSERDY` , `PLLRDY` , ...). After enabling, you must **poll the ready flag before using it** — switching SYSCLK to a not-yet-stable PLL hangs or faults.

2. The PLL

Q: How is the F7 main PLL configured?

Chain of dividers/multipliers (`RCC->PLLCFGR`):

```
source → /M → ×N (VCO) → /P → SYSCLK
                       → /Q → 48 MHz domain (USB/SDMMC/RNG)
```

- **/M**: bring the PLL input into the recommended ~1–2 MHz range.
- **×N**: VCO multiply; VCO output must stay within the datasheet window (~100–432 MHz).
- **/P**: divide VCO down to SYSCLK (≤ 216 MHz on F746).
- **/Q**: produce the 48 MHz peripheral clock.

Gotcha: keep the PLL input near **2 MHz** (set M accordingly) and the VCO inside its window, or the PLL won't lock / is out of spec. The intermediate stages have hard limits the final frequency alone doesn't reveal.

3. The clock tree (source → peripherals)

Q: Trace the clock from SYSCLK to a peripheral.

```
SYSCLK → AHB prescaler → HCLK (core, memory, DMA, SysTick base)
HCLK   → APB1 prescaler → PCLK1 (low-speed peripherals)   → APB1 timers (often ×2)
HCLK   → APB2 prescaler → PCLK2 (high-speed peripherals)   → APB2 timers (often ×2)
```

- **HCLK** clocks the core, the bus matrix, memories, and DMA.
- **PCLK1/PCLK2** clock peripherals; APB1 is the lower-max-frequency bus.
- **Timer clock = 2× PCLKx when that APB prescaler ≠ 1** (cross-ref timer notes — this is the recurring "frequency is 2× off" trap).

Q: How do you enable a peripheral's clock, and why does it matter?

Set the bit in the relevant `RCC->AHBxENR / APBxENR`. **A peripheral is electrically dead until its clock is gated on** — the #1 "my UART/SPI/GPIO does nothing" cause. Often needs a dummy read of the ENR after setting. There are also `…LPENR` registers controlling whether each peripheral clock keeps running in **Sleep** mode (clock-gating for power).

MCO pins can output a chosen internal clock (HSE/PLL/SYSCLK ÷ prescaler) — useful to verify on a scope what your clock *actually* is during bring-up.

---

4. CSS — Clock Security System

Q: What is CSS and why does it exist?

Enabled via `RCC->CR` CSSON. If the **HSE fails** (crystal dies, broken trace) while in use, the CSS **hardware-switches the clock back to HSI** and raises a **non-maskable interrupt (NMI)**. Without it, a dead crystal = a dead, unrecoverable board.

Gotcha: you must handle the CSS event in the **NMI handler** — clear the flag and decide what to do (limp on HSI, log, reset). People enable CSS and forget the NMI handler, so the fault path is untested. There's also a separate CSS for the LSE (RTC clock).

---

5. Voltage regulator & scaling (how you reach 216 MHz)

Q: What is voltage scaling (VOS) and over-drive?

The internal regulator powers the core; its voltage trades **power vs maximum frequency**:

- **VOS Scale 3** — lowest voltage, lowest max clock, least power.
- **VOS Scale 1** — highest voltage, highest max clock.
- On the **F746, reaching 216 MHz also needs Over-Drive mode**, on top of Scale 1.

Over-drive sequence (order matters): select VOS Scale 1 → enable Over-Drive (`PWR->CR1` ODEN, wait `ODRDY`) → enable Over-Drive switching (ODSWEN, wait `ODSWRDY`) → only then raise the clock.

Gotcha: running a high clock at too low a VOS scale (or without over-drive) is **out of spec** — unstable, intermittent faults. And remember the **Flash wait-state ordering** from the memory notes: raise wait states *before* raising the clock. Clock-up sequence on F7 is: WS up → VOS/over-drive → PLL up → switch SYSCLK. Get the order wrong and you corrupt fetches / HardFault.

Hardware note: the internal regulator needs external **VCAP** capacitors; VDDA (analog) and VBAT (backup) are separate supply pins.

---

6. Reset & supervision: POR, BOR, PVD

Q: What are the reset sources and how do you tell which one fired?

`RCC->CSR` latches the cause: **POR** (power-on), **BOR** (brown-out), **PIN** (NRST), **SFT** (software, via `NVIC_SystemReset()`), **IWDG/WWDG** (watchdog), **LPWR** (illegal low-power entry). Read and **clear these flags early in startup** (set `RMVF`) to know *why* you rebooted — invaluable for field debugging.

Q: BOR vs PVD — both watch the supply, what's the difference?

- **BOR (Brown-Out Reset):** if VDD drops below a configured threshold (level set in **option bytes**), the chip is **held in reset** — prevents executing on a sagging supply (which causes garbage/Flash corruption). It's a *reset*, automatic, no code.
- **PVD (Programmable Voltage Detector):** an **interrupt** (via EXTI line 16) when VDD crosses a programmable threshold, while still running. Used to **react** — flush data to Flash, park actuators — *before* a real brown-out hits. It informs; BOR enforces.

Gotcha: with BOR disabled/too low, a slow supply sag lets the CPU run on bad voltage and corrupt Flash during a write. For anything writing Flash, set a sensible BOR level.

---

7. Low-power modes (detail)

Q: Compare Run / Sleep / Stop / Standby on what's alive and how you wake.

Mode	Core	Clocks	RAM/regs	Regulator	Wake source	Wake speed
Run	on	on	on	on	—	—
Sleep	clock-gated	peripherals on	retained	on	any interrupt	instant
Stop	off	HSI/HSE/PLL off (LSI/LSE may stay)	retained	main or low-power reg	EXTI / RTC / wake pin	µs–ms
Standby	off	off	**lost** (except backup + standby RAM)	off	wake pin / RTC / IWDG / NRST	≈ reset

- **Entering:** `WFI` (wake on interrupt) or `WFE` (wake on event) after setting mode bits in `PWR->CR1` (e.g. PDDS, LPDS) and `SCB->SCR` (SLEEPDEEP for Stop/Standby).
- **Stop** can keep Flash in power-down and use the low-power regulator for less current, at the cost of slower wake.
- **Standby** loses RAM — on wake you start almost from reset; use **backup SRAM / RTC backup registers** to carry state across.

Gotchas: - Forgetting **SLEEPDEEP** ⇒ "Stop" actually just sleeps (no real savings). - A pending/unmasked interrupt makes `WFI` **return immediately** — looks like "sleep doesn't work." - **Debuggers drop the connection through Stop/Standby** (clocks/power gated) — expected, not a bug; use the debug-in-low-power config bits if you must keep SWD alive. - Configure wake sources **before** entering, and clear their pending flags, or you wake instantly or never.

---

8. Interview rapid-fire

1. **Why poll a clock ready flag?** Switching to an unstable source (PLL/HSE) hangs/faults.
2. **What does CSS do on HSE failure?** HW switches to HSI and fires an NMI.
3. **What extra does the F746 need for 216 MHz?** VOS Scale 1 + Over-Drive mode.
4. **Correct clock-up order on F7?** Flash wait-states up → VOS/over-drive → PLL → switch SYSCLK.
5. **BOR vs PVD?** BOR holds in reset below a threshold; PVD interrupts so software can react first.
6. **Where do you read the reset cause?** `RCC->CSR` flags (POR/BOR/PIN/SFT/IWDG/WWDG/LPWR).
7. **What's lost in Standby and how do you keep state?** RAM is lost; use backup SRAM / RTC backup registers.
8. `WFI` **returns immediately — why?** A pending unmasked interrupt; clear it / mask appropriately.
9. **Why is my peripheral completely dead?** RCC clock-enable bit not set.
10. **Why does Stop mode not save power?** SLEEPDEEP not set, so it only sleeps.
11. **What does MCO give you during bring-up?** A pin output of an internal clock to scope-verify the real frequency.

---

9. Daily-snippet candidates

- Bring up 216 MHz from a HSE crystal: configure M/N/P, set wait states, do the over-drive sequence, switch SYSCLK, verify via MCO.
- Enable CSS, write an NMI handler that detects HSE loss and limps on HSI.

- Read and decode `RCC->CSR` reset flags at boot; print the reset cause over UART, then clear.
- Configure PVD at a threshold, react in its EXTI ISR; configure BOR via option bytes.
- Enter Stop mode, wake on an RTC alarm; measure current before/after to confirm savings.

Deepen later from the STM32F7 reference manual RCC + PWR chapters and the datasheet's electrical/VOS frequency tables; AN4839 and the power-control app notes for low-power detail.

Part 7 — On-Chip Comm Peripherals (Config Side)

Microcontrollers — On-Chip Comm Peripherals (Config Side)

> Question-first notes. **Scope: the MCU-side peripheral blocks — registers, clocking, DMA/interrupt pairing, and bring-up gotchas.** Protocol theory (frame formats, addressing, arbitration, signal levels) lives in the **Communication Protocols** file. STM32F746 as the concrete example.

0. The universal bring-up pattern

Every on-chip comm peripheral follows the same config skeleton — internalize this and each block is just "fill in the specifics":

1. **Enable the peripheral clock** (`RCC->APBxENR`) — dead without it.
2. **Configure the GPIO pins**: AF mode + correct AF number; open-drain for I2C, push-pull for SPI/UART.
3. **Configure the peripheral**: speed/baud/timing, frame params, master/slave.
4. **Wire DMA** (optional): pick the datasheet-mapped stream/channel, set directions.
5. **Enable interrupts** in the peripheral + NVIC.
6. **Enable the peripheral** (the "EN" bit) — usually last.

1. USART / UART

Q: What configures a USART block (config side)?

- **Baud:** `BRR` derived from PCLK and the **oversampling** mode (×16 default, ×8 for higher baud at a given clock). `BRR = fck / baud` (with the oversampling factor folded in).
- **Frame:** word length (8/9 incl. parity), stop bits, parity enable/selection — in `CR1` / `CR2`.
- **Enable:** TE/RE (transmit/receive enable), UE (USART enable).
- **DMA:** set DMAT/DMAR (`CR3`) and point DMA at the data register; ideal for bulk TX/RX.
- **Key flags/IRQs:** TXE (data reg empty), RXNE (data ready), TC (transmission complete), **IDLE** (line idle — see gotcha), ORE (overrun).

Gotchas (config): - **Baud error %:** the integer `BRR` divisor rarely hits the exact baud; error must stay roughly **< 2 %** end-to-end or framing breaks — especially at high baud / odd PCLK. Check the computed actual baud, don't assume. - **IDLE-line interrupt** is the clean way to receive **variable-length** packets via DMA: DMA fills a buffer, the IDLE IRQ fires when the line goes quiet, you read how many bytes DMA wrote. Without it you're stuck guessing lengths. - **Overrun (ORE):** if you don't read RXNE fast enough (or via DMA) you lose bytes; must clear ORE or RX stalls. - Clock + GPIO AF mistakes are the usual "TX silent" cause (cross-ref GPIO notes).

(Protocol: start/stop/parity framing, RS-232/RS-422/RS-485 levels, your MAX490E RS-422 wiring → comms file.)

2. SPI

Q: What configures the SPI block (config side)?

- **Clock mode: CPOL/CPHA** in `CR1` (the four SPI modes). Must match the slave — your ADS131M08 is **mode 1 (CPOL=0, CPHA=1)**.
- **Baud:** `BR[2:0]` prescaler off PCLK (fPCLK/2 … /256).

- **Master/slave**, bit order (MSB/LSB first), and **data size** (4–16 bits; F7 SPI supports up to 16-bit frames, set in `CR2` DS).
- **NSS management:** hardware (NSS pin) vs **software** (SSM + SSI) — for multiple slaves you usually drive each CS as a plain GPIO and use software NSS.
- **F7 FIFO:** the F7 SPI has an RX/TX FIFO; set **FRXTH** (RX FIFO threshold) for byte vs half-word reads — wrong threshold ⇒ RXNE never asserts on single-byte transfers.
- **DMA:** TX/RX DMA via `CR2` TXDMAEN/RXDMAEN — essential for your high-rate streaming.

Gotchas (config): - **Wrong CPOL/CPHA** = garbage/shifted data — first thing to check against the slave datasheet. - **FRXTH** on F7: forgetting to set it for 8-bit access leaves bytes stuck in the FIFO. - **NSS/CS timing:** with software NSS you control CS edges — relevant to your ADS131M08 *frame boundary* requirement (CS must pulse between frames). The CS *behavior* is protocol; the *mechanism* (drive CS as GPIO via BSRR) is config — cross-ref GPIO BSRR note. - **Overrun (OVR)** in RX if you don't drain fast enough without DMA.

(Protocol: framing, your 24-bit/10-word/30-byte ADS131M08 frame, CS-edge boundary rule → comms file.)

---

3. I2C

Q: What's different about configuring the F7 I2C block?

The F7 uses the **"v2" I2C peripheral** — there is **no CCR/TRISE** like older STM32. Instead you program a single `TIMINGR` register (SCL prescaler, SCL high/low, setup/hold times).

- `TIMINGR` is normally **computed by STM32CubeMX or the ST tool** for your PCLK + target speed (Standard 100k / Fast 400k / Fast-mode-plus 1M) — hand-calculating it is error-prone.
- **Own address(es)** for slave mode, addressing mode (7/10-bit).
- **Analog + digital noise filters** (configurable) on SDA/SCL.
- **DMA** for TX/RX; interrupts for TXIS/RXNE/STOPF/NACKF/errors.
- **GPIO must be open-drain with pull-ups** (cross-ref GPIO) — non-negotiable for I2C.

Gotchas (config): - **TIMINGR wrong** ⇒ out-of-spec SCL timing, marginal/failed comms. Use the tool's value for your exact PCLK. - **Clock stretching** must be allowed (don't disable it) for slaves that need it. - Missing external pull-ups (or relying on weak internal ones) ⇒ slow edges, stuck bus. - A bus left stuck-low (slave mid-transaction at reset) may need a **manual 9-clock bus-recovery** toggle on SCL before re-init.

(Protocol: addressing, ACK/NACK, start/stop/repeated-start, arbitration → comms file.)

---

4. CAN (bxCAN) — relevant to your DRDO/CAN work

Q: What configures the bxCAN block (config side)?

- **Bit timing** (`BTR`): **BRP** (baud prescaler), **TS1/TS2** (time segments), **SJW** (resync jump width). These set the bit rate *and* the **sample point** — the fraction of the bit where the level is read (target ~75–87.5 %). `bit_rate = PCLK / (BRP × (1 + TS1 + TS2))`.
- **Modes:** Normal, **Loopback** (self-test, no bus), **Silent** (listen-only, no ACK), or Loopback+Silent. Plus the mandatory **Initialization mode** transition (INRQ/INAK) to configure.
- **Filters:** acceptance filters (mask or list mode, 16/32-bit) routing IDs to **FIFO0/FIFO1**; unconfigured filters = you receive nothing.
- **Mailboxes:** 3 TX mailboxes, 2 RX FIFOs; DMA isn't used (mailbox/FIFO based), interrupts on RX-pending/TX-complete/errors.

Gotchas (config): - **Sample point / bit timing miscalculation** is *the* CAN bring-up bug — both nodes must agree on bit rate **and** sit at a compatible sample point, or you get error frames / bus-off. Use a CAN bit-timing calculator for BRP/TS1/TS2/SJW at your PCLK. - **No RX filter configured** ⇒ silence (everything

filtered out). - Forgetting to **leave Init mode** ⇒ peripheral never goes on-bus. - A lone node with no other ACKing node → TX error counter climbs → **bus-off** (use Loopback to test solo).

(Protocol: arbitration, IDs, DLC, stuffing, error frames, your predictive-maintenance CAN payloads → comms file.)

---

5. Higher-bandwidth peripherals (config highlights)

Q: Quick config-side notes on USB / Ethernet / SDMMC / I2S?

- **USB (OTG_FS/HS):** needs an accurate 48 MHz clock (from PLL /Q or a dedicated PLL); HS needs a ULPI PHY. Endpoint/FIFO config is heavy — almost always via the ST USB stack/middleware.
- **Ethernet MAC:** uses **DMA descriptor rings** for RX/TX buffers in RAM (so cache coherency on F7 applies — descriptors/buffers in non-cacheable or cleaned/invalidated SRAM). MII/RMII selected in SYSCFG; needs an external PHY + 25/50 MHz clock.
- **SDMMC:** clock divider for card speed, 1-/4-bit bus width, **DMA** for block transfers; respects the same cache rules for its DMA buffers.
- **I2S** (often shared with SPI block): audio clocking from a dedicated **PLLI2S** to hit exact sample-rate master clocks; DMA-fed.

Common thread: the fast peripherals all lean on **DMA + correct clocking**, and on F7 they all hit the **cache-coherency** rule for their RAM buffers (cross-ref Memory/DMA notes).

---

6. Interview rapid-fire (config-focused)

1. **Universal first step for any comm peripheral?** Enable its RCC clock; then GPIO AF.
2. **Max acceptable UART baud error, roughly?** ~< 2 %; check actual vs target divisor.
3. **Cleanest way to DMA-receive variable-length UART packets?** IDLE-line interrupt + DMA byte count.
4. **What must match between SPI master and slave?** CPOL/CPHA (mode), bit order, data size.
5. **F7 SPI single-byte RX stuck — likely cause?** FRXTH FIFO threshold not set for 8-bit access.
6. **How does F7 I2C set timing (vs older STM32)?** Single TIMINGR register (tool-computed), not CCR/TRISE.
7. **Mandatory GPIO config for I2C?** Open-drain + pull-ups.
8. **What three things set CAN bit timing?** BRP, TS1/TS2, SJW (defining rate + sample point).
9. **CAN node receives nothing — config cause?** No acceptance filter configured.
10. **How to bench-test CAN with one node?** Loopback mode (no external ACK needed).
11. **Why do Ethernet/SDMMC DMA buffers need care on F7?** D-cache coherency — non-cacheable or clean/invalidate.

---

7. Daily-snippet candidates

- UART RX of variable-length frames using DMA + IDLE-line interrupt; print received length.
- Bare-metal SPI master read of an ADS131M08 register (mode 1, software NSS via GPIO BSRR).
- I2C scan of the bus using a tool-computed TIMINGR; list responding addresses.
- bxCAN in loopback: configure BTR for a target rate/sample point, send + receive a frame solo.
- Ethernet/SDMMC: set up a DMA descriptor/buffer in non-cacheable SRAM and confirm a transfer.

Deepen later from the STM32F7 reference manual USART/SPI/I2C/CAN chapters, ST's I2C TIMINGR app note, and a CAN bit-timing calculator. Protocol-level depth → the Communication Protocols file.

Part 8 — Development & Tooling

Microcontrollers — Development & Tooling

> Question-first notes. The workflow layer that ties the peripheral files together: which abstraction to code against, how the build actually works, how to debug on hardware, and how firmware update / boot works. STM32F746 as the concrete example.

---

1. HAL vs LL vs CMSIS vs raw registers

Q: What are the abstraction layers and when do you use each?

Layer	What it is	Pros	Cons	Use when
HAL	ST's high-level driver (`HAL_UART_Transmit`)	Fast to write, portable across STM32, handles edge cases	Bloated, opaque, hidden timeouts/state, hard to time precisely	Prototyping, complex peripherals (USB, Ethernet), non-critical paths
LL (Low-Layer)	Thin inline wrappers over registers (`LL_USART_TransmitData8`)	Near-register speed, readable, still some safety	Less hand-holding, you manage sequencing	Performance-sensitive paths, when HAL is too heavy but you want names not magic numbers
CMSIS	ARM's core definitions (`SCB->`, `NVIC_*`, intrinsics) + device register structs	Vendor-neutral core access, always available	Core/register only — no peripheral *drivers*	Always present underneath; use directly for core (NVIC/SCB/SysTick)
Raw registers	Write `RCC->APB1ENR |= ...` yourself	Total control, smallest, you understand every bit	Verbose, error-prone, you own every gotcha	Learning, ISRs/hot loops, when you must know exact timing/behavior

Q: What's the practical recommendation?

Mix layers. A common professional pattern: **CMSIS for the core**, **LL or raw registers** for the hot, timing-critical peripherals (your SPI/ADC/DMA DAQ path), and **HAL** for the heavy stuff you don't want to hand-roll (USB, Ethernet, SDMMC/FatFS). Don't be dogmatic — "all HAL" wastes cycles and hides bugs; "all registers" wastes your life on USB.

Gotcha (interview-relevant): the reason to know raw registers isn't purity — it's that when HAL misbehaves (a silent timeout, a state-machine stall), you can only debug it if you understand the registers underneath. Your Feb gap "register-level drivers without HAL" is exactly this skill, and you've since closed it on the ADS131M08 driver — that's a strong talking point.

---

2. The GCC toolchain (arm-none-eabi)

Q: What are the toolchain pieces and what does each do?

- `arm-none-eabi-gcc` — compiler/driver. "none-eabi" = bare-metal (no OS), ARM EABI.
- `arm-none-eabi-ld` — linker (usually invoked via gcc) — places sections per the linker script.
- `arm-none-eabi-objcopy` — convert the ELF to `.bin` / `.hex` for flashing.
- `arm-none-eabi-objdump` — disassemble / inspect sections.
- `arm-none-eabi-size` — report `.text` / `.data` / `.bss` sizes (does it fit in Flash/RAM?).

- `arm-none-eabi-gdb` — debugger front-end (talks to OpenOCD/ST-Link/J-Link).
- `arm-none-eabi-nm` / `readelf` — symbols / ELF detail.

Key compile flags for Cortex-M7:

```
-mcpu=cortex-m7 -mthumb
-mfpu=fpv5-d16 -mfloat-abi=hard      # F746 double-precision FPU, hardware float ABI
-Og -g                               # debug build: light opt + symbols
-Os                                  # release: optimize for size (typical embedded)
-ffunction-sections -fdata-sections  # + linker --gc-sections to drop unused code
-Wall -Wextra
```

Gotcha #1 — float ABI mismatch: every object **and** the libraries must agree on `-mfpu` / `-mfloat-abi`. Mixing soft-float and hard-float objects → cryptic link errors or wrong results. Set them globally.

Gotcha #2 — `-O0` debugging surprises: at `-O0` everything is literal (easy stepping) but big and slow; at `-O2/-Os` the compiler reorders/inlines and **a missing `volatile` finally bites** (see core notes). Bugs that "only appear in release" are usually `volatile`/timing/UB. Use `-Og` for a sane debug experience.

---

3. The linker script & build glue

Q: What does the linker script do (recap + build view)?

Defines memory regions (FLASH/DTCM/SRAM) and maps sections (`.text`, `.data`, `.bss`, stack, heap) into them — cross-ref Memory notes. The **startup file** (`startup_stm32f746.s`) provides the vector table, `Reset_Handler` (the `.data` copy / `.bss` zero loop), and default handlers.

Q: Make vs CMake vs IDE?

- **Makefile** — explicit, transparent, what most ST examples and CI use. You see every flag.
- **CMake** — scales better for multi-target/multi-lib projects; increasingly the STM32 default.
- **IDE-managed (CubeIDE)** — generates the build for you; convenient but hides flags (and can clobber hand edits on regeneration — keep custom code in the USER CODE guard blocks).

A minimal flow: compile each `.c` → link with the `.ld` script + startup → `objcopy` to `.bin` → flash. **Gotcha:** the linker `.map` file is your friend — check it to confirm sizes and **where each symbol/buffer actually landed** (the DTCM-vs-SRAM DMA trap shows up here).

---

4. Flashing & on-chip debug (SWD)

Q: How do you get code onto the chip and debug it?

- **SWD** (2-wire: SWDIO + SWCLK, + GND, + optionally SWO) is the standard debug interface — see core notes. Probes: ST-Link, J-Link, CMSIS-DAP.
- **Flashing tools:** OpenOCD or ST-Link tools / `STM32_Programmer_CLI`, driven from the IDE or CLI.
- **Debug session:** GDB connects through OpenOCD/J-Link GDB server to load, run, breakpoint, step, inspect memory/registers.

Q: Hardware breakpoints vs watchpoints — what's the difference and the limit?

- **Breakpoint** — halt when the **PC reaches an address** (a line of code). Cortex-M has a small number of **hardware breakpoint** comparators (e.g. ~6–8 on M7 via the FPB unit). Exceed them and the debugger either refuses or falls back to slow software breakpoints (only usable in RAM).
- **Watchpoint** (data watchpoint, via the **DWT** unit) — halt when a **memory address is read/written** (or matches a value). Even fewer comparators (often ~4). Invaluable for "who is corrupting this variable?" — set a write watchpoint on the clobbered global and catch the culprit.

Gotcha: running out of hardware breakpoints is common; software breakpoints don't work in Flash (can't rewrite it live), so over-breakpointing Flash code fails. Be economical, or use watchpoints for data bugs.

Q: printf-debugging without a UART?

- **ITM/SWO** — route `printf` over the single SWO trace pin (core notes); fast, doesn't need a free UART.
- **Semihosting** — `printf` through the debugger to the host console, but it **halts the core on each call** — fine for bring-up, **never** in timing-sensitive or production code.
- **DWT cycle counter** (`DWT->CYCCNT`) — free-running CPU cycle counter for precise profiling of a code section (great for timing your DAQ ISR).

---

5. CRC peripheral

Q: What is the on-chip CRC unit and why use it?

A hardware CRC calculator (the F7's is **configurable**: polynomial, initial value, input/output bit reversal, byte/half-word/word feed). Feed data into `CRC->DR`, read the result out.

Uses: integrity-check a firmware image or a config block, validate received packets, fast checksums without burning CPU.

Gotchas: - **Default polynomial** on classic STM32 CRC is the fixed Ethernet poly `0x04C11DB7`, **MSB-first, no reversal** — to match a standard like **CRC-32 (zlib)** you must set input/output **bit reversal** and the right init value, or your hardware CRC won't match software/PC-side CRCs. Matching a PC implementation is the classic CRC headache. - It's **word-fed**: byte ordering/endianness of how you push data affects the result — be consistent between the MCU and whatever checks the CRC.

---

6. Bootloaders & IAP (in-application programming)

Q: What's the difference between the built-in ROM bootloader and a custom bootloader/IAP?

- **System (ROM) bootloader** — burned into system memory by ST; entered via **BOOT pins / option bytes**. Talks UART/USB-DFU/etc. to flash the chip with no debugger. Good for factory programming and recovery.
- **Custom bootloader / IAP** — *your* code in the lower Flash sectors that can reprogram the application sectors (firmware update over UART/CAN/USB/OTA), then jump to the app.

Q: How does a custom bootloader hand off to the application?

1. Bootloader validates the app (e.g. CRC check, magic number).
2. **Set the new vector table base**: `SCB->VTOR = APP_ADDR;`
3. Load the app's **initial MSP** from `*(APP_ADDR)` and set `__set_MSP()`.
4. Read the app's **reset vector** from `*(APP_ADDR + 4)` and jump to it.

```c
void jump_to_app(uint32_t app_addr) {
    uint32_t msp   = *(volatile uint32_t *)(app_addr);
    uint32_t reset = *(volatile uint32_t *)(app_addr + 4);
    __disable_irq();
    SCB->VTOR = app_addr;                // relocate vector table
    __set_MSP(msp);                      // set app stack pointer
    ((void (*)(void))reset)();           // jump to app reset handler
}
```

Gotchas (these are the real bootloader bugs): - **Forgetting** `SCB->VTOR` ⇒ the app's interrupts vector into the *bootloader's* handlers (core notes). Classic. - **Not de-initializing** what the bootloader enabled (clocks, peripherals, **SysTick**, interrupts) before the jump ⇒ the app inherits a dirty state and faults. Disable IRQs and reset peripherals first. - **Linker mismatch:** the app must be **linked to its real Flash origin** (e.g. `0x08008000`), not `0x08000000`, or its absolute addresses are wrong. - **Flash rules apply** (Memory notes): can't execute from the sector being erased; respect sector granularity; the bootloader's reprogram routine may need to run from RAM. - **Dual-bank** F7 parts let you program the inactive bank while running from the active one — basis for safe A/B OTA updates.

7. Interview rapid-fire

1. **When drop from HAL to LL/registers?** Timing-critical/hot paths, or when HAL hides a bug you must see.
2. **Why know registers even if you use HAL?** To debug HAL when its state machine/timeout misbehaves.
3. **F746 float flags?** `-mfpu=fpv5-d16 -mfloat-abi=hard`; all objects + libs must match.
4. **Why does a bug appear only in release builds?** Usually a missing `volatile`, UB, or timing exposed by optimization.
5. **What does `objcopy` do in the build?** ELF → `.bin` / `.hex` for flashing.
6. **Breakpoint vs watchpoint?** BP halts on PC=address; WP halts on data access — limited HW comparators each.
7. **How to find what corrupts a variable?** Set a data write watchpoint (DWT) on its address.
8. **Why won't software breakpoints work in Flash?** Can't rewrite Flash live; SW BPs need RAM.
9. **Why doesn't my HW CRC match the PC's CRC-32?** Bit-reversal/init/poll settings differ — configure reversal + init.
10. **Two must-dos when jumping bootloader→app?** Set `SCB->VTOR`; set app MSP, then jump to its reset vector (after disabling IRQs/deinit).
11. **Where must the app be linked for a bootloader at 0x08000000?** Its own offset (e.g. `0x08008000`), matching VTOR.
12. **Free way to profile a code section's cycles?** `DWT->CYCCNT`.

8. Daily-snippet candidates

- Write a Makefile from scratch for an F746 project (compile, link with `.ld`, objcopy, flash, `size`).
- Same blink in three layers — raw registers, LL, HAL — and compare the `.text` size with `arm-none-eabi-size`.
- Use a DWT watchpoint to catch a deliberately corrupted global.
- Profile an ISR with `DWT->CYCCNT`; print cycles over SWO/ITM.
- Configure the CRC unit with reversal to match Python's `zlib.crc32`, verify they agree.
- Minimal two-image setup: a bootloader that CRC-checks and jumps to an app linked at an offset (set VTOR/MSP).

Deepen later from the GNU Arm Embedded toolchain docs, the STM32F7 reference manual CRC chapter, ST's AN2606 (system bootloader) + AN4657/IAP app notes, and the OpenOCD/GDB manuals.

✅ Microcontroller subject — complete

Files 01–08 now cover: core/ARM Cortex · GPIO & EXTI · timers/PWM/watchdog/RTC · ADC/DAC · memory/DMA/Flash · clock & power · comm-peripheral config · dev & tooling. Protocol *theory* is intentionally deferred to the Communication Protocols subject. Next step is either deepening the heavy hitters (core, DMA, ADC) from a book, or starting the next subject.